Math 40a: Introduction to Applied Mathematics
Lab 4

Upload your write-up and Python files to Moodle to submit your assignment. The write-up should be a self contained pdf—the graders should be able to evaluate your lab based on the write-up without having to look at your separate code files. If you choose to upload a pdf of an IPython notebook, it is okay if your code is included within the writeup, but please make sure to include clear answers to the questions.

Overview

Today's computer lab has two aims:

1. To run simulations of the SIR (Susceptible–Infected–Recovered) model introduced in class and explore simple aspects of this model.

2. To download Google Flu data and compare it to the predictions of the SIR model.

Getting Started

In the SIR model discussed in class, there are three populations, the susceptibles $S(t)$, the infecteds $I(t)$ and the recovered $R(t)$. The dynamics are given by the system of ordinary differential equations,

$$\begin{aligned}\frac{dS}{dt} &= -\beta SI \\ \frac{dI}{dt} &= \beta SI - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

where β is the rate at which an infected person infects a susceptible and γ is the recovery rate of infected people.

The Python code needed to solve these equations is provided below, which makes use of the `odeint` function within the SciPy library to do the numerical integration. Save this code in a file `sir.py`.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
beta=1.0/1000
gamma=1.0/(24*14)
def deriv(x,t):
    ifc=beta*x[0]*x[1]
    rec=gamma*x[1]
    return np.array([-ifc,ifc-rec,rec])
time=np.linspace(0,2000,1000)
xinit=np.array([100,1,0])
x=odeint(deriv,xinit,time)
plt.figure()
p0,=plt.plot(time,x[:,0])
p1,=plt.plot(time,x[:,1])
p2,=plt.plot(time,x[:,2])
plt.legend([p0,p1,p2],["S(t)","I(t)","R(t)"])
plt.xlabel('t (Hours)')
plt.ylabel('Population')
plt.show()
```

The `deriv` function encodes the SIR model dynamics. For a given input array x with three elements corresponding to S, I, and R, the function calculates dS/dt, dI/dt, and dR/dt. The `odeint` function takes three arguments: the time array that provides a list of timepoints at which to calculate the solution, the `xinit` array that provides the initial values of the three variables, and a reference to the `deriv` function.

1. Write a comment next to each line of the SIR code to explain to another human (and yourself!) what that line does.

2. Run this program, and plot $S(t)$, $I(t)$ and $R(t)$ for this set of parameters. Now find the initial value of $S(0)$ so that the flu epidemic does not grow. Compare this to the threshold value discussed in the SIR paper and in class.

3. Rewrite your program so that instead of measuring time in hours, time is measured in weeks. This requires rewriting your rate constants so that they are expressed in units of week^{-1} instead of hour^{-1}. Remember to modify your axes labels!

Google Flu Data

We now want to read in data from Google Flu and examine it. A version of the Google Flu data (`googleflu.csv`) is available on the course website.[1] It is aggregated on a weekly basis. The matrix has 53 columns listed by state, with the columns corresponding to Date, United States, Alabama, Alaska, Arizona, Arkansas, California, Colorado, Connecticut, Delaware, District of Columbia, Florida, ..., Wisconsin, Wyoming. Hence the first column is the "Date", the second column is the United States, and Massachusetts is column number 24.

To read the file into Python, save it into the directory in which you are working, and then make use of the following program, `parse.py`.

```
import numpy as np
import matplotlib.pyplot as plt
# Open the file in the "read" mode
f=open("googleflu.csv","r")

# Read in header line
h=f.readline().split(",")
# Initialize 53 empty lists
s=[]
for i in range(53):
    s.append([])
# Loop over the remaining lines
for l in f:
    # Create a list by separating the line at commas
    d=l.split(",")
    for i,val in enumerate(d):
      if val=='' or val.isspace():
        d[i] = np.NaN

    # Store the 53 entries in the line to each relevant list
    for i in range(53):
      if i==0:
          s[i].append(d[i])
        else:
          s[i].append(float(d[i]))

# Close the file
f.close()
```

[1] If you open this or other csv files in Excel and it gets saved, it may get corrupted. If you have problems, re-download the file and try again.

```
plt.figure()
p3,=plt.plot(s[1][0:52],'-')
plt.legend([p3],["Data"])
plt.xlabel('t (Weeks)')
plt.ylabel('Population')
plt.show()
```

This program reads the data into 53 separate lists. Currently the program focuses on the United States data in the first year by plotting the first 52 entries of the `s[1]` array, which is stored as `s[1][0:52]`.

4. Plot the first year of the data for the United States. Try to find parameters of the SIR model such that the model fits the data well. Play around with it until you find something that works. Are the parameters reasonable given what you know about the flu?

Finding a good fit is hard. Focus on trying to fit a few aspects of the graph:

(a) The maximum number of infected people. This happens when $dI/dt = 0$ or $S = \gamma/\beta$.

(b) The timescale for getting to this maximum.

(c) The timescale for recovery.

Think about these and discuss them with your peers and instructor.